

Application of LLL for broke instance of ECDSA

Edoardo Torrini

May 2026

1 Elliptic Curve Digital Signature Algorithm (ECDSA)

ECDSA domain parameters

Define Elliptic Curve E defined over $\mathbb{Z}_p$ (es. **P-256**¹), with $n = \#E(\mathbb{Z}_p)$ where n is the order group and a prime number.

A generator $P \in E(\mathbb{Z}_p)$ and a **collision-resistance hash function** H whose output has the same **bitlength** as n (es. $H = \text{SHA256}$)

ECDSA key generation

Alice does:

1. select $a \in_R [1, n - 1]$ and compute $A = aP$.
2. Alice's **public key** is A ; her **private key** is a

Note: computing a from A is an instance of the **ECDLP**.

ECDSA signature generation

To sign a message $M = \{0, 1\}^*$, Alice does:

1. compute $m = H(M)$ and convert m as an integer.
2. select a **ephemeral key per-message secret** $k \in_R [1, n - 1]$.
3. compute $R = kP$. Let $r = x(R) \pmod{n}$, and check that $r \neq 0$ ($x(R)$ is the x-coordinate of R).
4. compute $s = k^{-1}(m + ar) \pmod{n}$ and check that $s \neq 0$.
5. Alice signature is (M, r, s) .

ECDSA verify signature

Bob knows the $(A, M, (r, s))$ and does:

1. verify that r and s are integers in $[1, n - 1]$. If not the signature is **invalid**.
2. compute $m = H(M)$, where H is the same hash function used in the signature generation.
3. calculate $u_1 = zs^{-1} \pmod{n}$ and $u_2 = rs^{-1} \pmod{n}$.
4. compute the curve point $(x_1, y_1) = u_1 \times P + u_2 \times A$ if $(x_1, y_1) = \mathcal{O}$ then the signature is **invalid**.
5. the signature is **valid** if $r \equiv x_1 \pmod{n}$.

Note: an efficient implementation would compute inverse $s^{-1} \pmod{n}$ only once. Also, using Shamir's trick, a sum of two scalar multiplications $u_1 \times P + u_2 \times A$ can be calculated faster than two scalar multiplications done independently².

¹<https://modern-cryptography.com/nist-recommended-elliptic-curves-p-256-p-384-and-the-standardization-of-secure-curves>

²https://www.lirmm.fr/~imbert/talks/laurent_Asilomar_08.pdf

2 Cryptanalyze ECDSA using LLL application

LLL is a very powerful cryptanalytic tool, and can be used to break ECDSA with partial information. The general methodology is:

1. define a lattice L in such way that the solution to the cryptanalytic problem is encoded in a nonzero lattice vector v of unusually small length.
2. run LLL algorithm to find a short basis B for L .
3. check whether v is in B .

In the ECDSA signature scheme, a signer has: a **long-term private key a** and a **per-message secret k** . Both must be generated *uniformly at random* from the private key space.

Is possible that some per-message secrets might be leaked during the signing process via a **side-channel attack**: poor implementation, weak random number generator, timing analysis, electromagnetic radiation analysis, etc. This side-channel information can sometimes be used to compute the long-term private key a .

2.1 Leakage: two signatures

Consider the scenario where partial information is leaked about the per-message secret for *two* signed messages. The knowledge of a signed message (M, r, s) yields a linear congruence in two unknowns: **$ks \equiv m + ar \pmod{n}$** . Suppose that an adversary obtains two signed messages (M_1, r_1, s_1) and (M_2, r_2, s_2) . This yields two linear congruences:

$$\begin{aligned} k_1 s_1 &\equiv m_1 + ar_1 \pmod{n} & k_2 s_2 &\equiv m_2 + ar_2 \pmod{n} \\ (k_1 s_1 - m_1) r_1^{-1} &\equiv a \pmod{n} & (k_2 s_2 - m_2) r_2^{-1} &\equiv a \pmod{n} \end{aligned}$$

Solving for a and equating return the congruence:

$$\begin{aligned} (k_1 s_1 - m_1) r_1^{-1} &\equiv (k_2 s_2 - m_2) r_2^{-1} \pmod{n} \\ (k_1 s_1 - m_1) r_2 &\equiv (k_2 s_2 - m_2) r_1 \pmod{n} \end{aligned}$$

Rearranging gives the **linear congruence**: **$k_1 - (s_1 r_2)^{-1} s_2 r_1 k_2 + (s_1 r_2)^{-1} (m_2 r_1 - m_1 r_2) \equiv 0 \pmod{n}$** where the only quantities that are not known to the adversary are k_1 and k_2 . One linear congruence in two unknowns does not usually sufficient to recover k_1 and k_2 . However it might be possible to solve k_1 and k_2 if the attacker already has substantial partial information about k_1 and k_2 . For example an upperbound on their size. If the adversary can solve for k_1 and k_2 , they immediately recover the long-term key a and thus the ECDSA instance is completely broken.

Let's assume that $k_1, k_2 \leq K$, where $K \ll n$. Is possible to rewrite the congruence as **$k_1 + tk_2 + u \equiv 0 \pmod{n}$** where $t = -(s_1 r_2)^{-1} s_2 r_1 \pmod{n}$ and $u = (s_1 r_2)^{-1} (m_2 r_1 - m_1 r_2) \pmod{n}$. Is possible to convert the congruence into the equation:

$$k_1 + tk_2 + hn + u = 0 \tag{1}$$

where $h \in \mathbb{Z}$. The adversary can compute t and u , and knows n .

Consider the lattice L with basis vectors **$(n, 0, 0)$, $(t, 1, K)$ and $(u, 0, K)$** . The basis matrix for L is:

$$B = \begin{bmatrix} n & t & u \\ 0 & 1 & 0 \\ 0 & 0 & K \end{bmatrix}$$

so the volume of L is $\text{vol}(L) = nK$. Note that:

$$v = B[h, k_2, 1]^T = \begin{bmatrix} n & t & u \\ 0 & 1 & 0 \\ 0 & 0 & K \end{bmatrix} \begin{bmatrix} x \\ k_2 \\ 1 \end{bmatrix} = \begin{bmatrix} hn + tk_2 + u \\ k_2 \\ K \end{bmatrix}$$

Compute row by row:

- Row 1: $n \cdot x + t \cdot k_2 + u \cdot 1 = nx + tk_2 + u$
- Row 2: $0 \cdot x + 1 \cdot k_2 + 0 \cdot 1 = k_2$
- Row 3: $0 \cdot x + 0 \cdot k_2 + K \cdot 1 = K$

We obtain the vector $v = [tk_2 + hn + u, k_2, K]^T$ if we use the equation 1 we can substitute the $tk_2 + hn + u$ with the terms $-k_1$ so it is possible to obtain the vector $[-k_1, k_2, K]$.

This lattice vector has length $\|v\| = \sqrt{k_1^2 + k_2^2 + K^2} \leq \sqrt{3K^2} = \sqrt{3}K$. The *Gaussian heuristic* asserts that $\lambda_1(L) \approx \sqrt{3/2\pi e}(\text{vol}(L))^{\frac{1}{3}} \approx (nK)^{\frac{1}{3}}$. So, if $\|v\| \ll (nK)^{\frac{1}{3}}$, we can expect that v is a shortest nonzero vector in L . In this event, the adversary can (likely) determine v by solving SVP in L . Knowledge of v immediately reveals k_1 and k_2 , and then the long-term private key a can be efficiently computed from the signing equation.

This requirements is satisfied if more than half of the leftmost bits of each per-message secret are known to be zero.